TAKE-HOME ENGINEERING EXERCISE

Event Intelligence Chat

Build a conversational feature for an existing event-intelligence product.

TIMEBOX	INPUTS	MODEL
One working day	30 synthetic events + 1 company profile	Hosted LLM/API of your choice

Background

Assume you are joining a team working on an existing strategic-intelligence product. The product already collects and stores external events and maintains basic context about the company using it. We want to add a chat feature that lets a user ask questions about those events in natural language.

For this exercise, you are given a small synthetic event dataset and one fictional company profile. Your task is to build the chat feature around those inputs. The event collection pipeline, data enrichment pipeline, user management and the rest of the product can be treated as already existing.

What we are interested in

We are interested in how you design the application around a capable hosted language model: how you construct context, ground answers in supplied data, maintain conversational continuity, expose supporting evidence, handle uncertainty and keep the implementation proportionate to the problem size.

1. Your task

Build a small standalone web application with a chat interface. The user should be able to ask questions about the supplied events and receive answers that take the fictional company context into account when relevant.

2. Core requirements

Conversational chat

Provide a usable chat interface and support follow-up questions that depend on earlier turns. For example, after asking for the most relevant risks, the user should be able to ask "Which of those could affect our cost base first?".

Grounded answers

Factual claims about events must be grounded in the supplied event dataset. The response should identify the supporting event IDs and/or source names so the user can see what the answer is based on.

Company-aware relevance

When the user asks what matters most, what creates a risk, or what could create an opportunity, use the supplied Asteron Systems profile as context. The company profile should influence relevance and prioritization rather than being repeated mechanically.

Facts vs. analysis

The assistant may interpret implications of an event for the company, but the response should make a reasonable distinction between what is explicitly stated in the data and what is an analytical inference.

Insufficient evidence

If the supplied information does not support an answer, say so. Do not fill gaps with confident assertions from general model knowledge. It is acceptable to explain what additional information would be needed.

Model integration

Use a mainstream hosted LLM/API that you are comfortable with. Keep provider credentials on the server side and document the model/provider choice and the main prompting/context decisions.

Failure handling

Handle at least basic failure modes gracefully - for example, a model/API error, an empty response, or a response that does not contain usable supporting references.

3. Context selection and scale

The supplied corpus contains only 30 events. Choose the simplest context-selection approach you believe is appropriate at this scale. Sending all events to the model, deterministic filtering, lexical search, embeddings, or another approach can all be reasonable depending on your design.

We care more about the quality of the decision than about using a particular retrieval technology. In your README, briefly explain what you would change if the system had 100,000+ events or received a large continuous event stream.

4. Scope

In scope	Out of scope
Chat UI and multi-turn conversation	Web scraping or live news ingestion
Use of supplied event/company context	Authentication, permissions or multiple user personas
Grounding and references to supporting events	Building the rest of the intelligence platform
Relevance/risk/opportunity reasoning	Deployment or production infrastructure
Basic model/API and parsing failure handling	Pixel-perfect visual design
A small set of repeatable tests or evaluations	A required vector database or retrieval framework
Clear technical explanation in the README	Any particular frontend/backend framework

5. Technical expectations

- Use any reasonable web stack. The repository should run locally from documented instructions.
- Keep LLM/provider credentials out of browser-delivered code.
- Keep the event/company context and model interaction understandable and maintainable. We should be able to follow how a user question becomes a grounded answer.
- Include a small number of meaningful tests or repeatable evaluation cases. These can be automated tests, an evaluation script, or another reproducible approach.
- Do not optimize for feature count. A smaller implementation with reliable core behavior is preferable to a large but fragile prototype.

6. Example questions

These are examples of the kinds of questions we may use when reviewing the exercise. You do not need to hard-code behavior for them.

- *What are the three developments most relevant to Asteron Systems right now?*
- *Which of those could affect our cost base within the next 12 months?*
- *Are there opportunities related to automation or AI?*
- *What evidence suggests cloud dependency is a strategic risk for us?*

- *Compare the semiconductor export-control event with the electricity-grid constraints. How could each affect Asteron?*
- *Which developments could create supply-chain resilience opportunities?*
- *Will interest rates fall next quarter?*

7. What we will look at

Grounding: Whether answers are actually supported by the supplied events and whether references are meaningful rather than decorative.

Technical judgment: Whether the solution matches the small exercise scale without unnecessary complexity, while showing awareness of how the architecture would evolve at scale.

AI application design: Prompt/context construction, conversational continuity, uncertainty handling, and how the model output is turned into a reliable product response.

Code quality: Clear boundaries between UI, server/model integration and domain logic; readable code; sensible error handling.

Company relevance: Whether the assistant uses the company context to prioritize and explain relevance rather than producing generic news summaries.

Evaluation mindset: Whether you have a practical way to check that the chat behaves correctly on representative questions and failure cases.

8. Deliverables

1. A Git repository containing the implementation.
2. A README with setup/run instructions.
3. A short architecture explanation, including the model/provider used and how event context is selected and supplied.
4. A short explanation of how answers are grounded and how insufficient evidence is handled.
5. A brief note on the main trade-offs you made because of the one-day timebox.
6. A short explanation of how you would adapt the approach for a much larger event corpus.
7. A few repeatable test/evaluation cases that demonstrate the core behavior.

Files supplied with this brief

1. event_intelligence_exercise_events.xlsx - synthetic global events
2. event_intelligence_exercise_company_context.xlsx - fictional company profile

9. Timebox and submission

Please treat this as a one-working-day exercise. If you reach the timebox before completing everything, stop and note what remains and what you would do next. We value prioritization and clear technical reasoning over trying to complete every possible enhancement.